Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Template for Master's Theses at DIBRIS

Matteo Dell'Amico

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/

Università
di Genova

Computer Science MSc
Software Security and Engineering Curriculum

Template for Master's Theses at DIBRIS

Matteo Dell'Amico

Advisor: Advisor Name

Examiner: Examiner Name

May, 2026

Abstract

This is a template for the Master's thesis in Computer Science at the University of Genoa.

This document doubles as a guide for how to structure and write it. Please keep a copy of this advice and read it carefully while you write your thesis by replacing it with your material.

Table of Contents

Chapter 1 Introduction 6

Chapter 2 Background 7

2.1 Windows Process Architecture and DLL Injection 7

2.2 The Application Binary Interface (ABI) and Calling Conventions 9

2.3 Execution Context and CPU State Preservation 9

2.4 Paradigms of Binary Instrumentation 10

2.5 Inline API Hooking Architectures 10

2.5.1 Bare-Metal Hooking: Microsoft Detours and MinHook 11

2.5.2 Context-Aware Routing: Frida-Gum 12

2.5.3 Passive vs. Active Trampoline Architectures 14

2.5.4 Instruction Boundaries and the Role of Disassemblers 15

2.5.5 Transactional Patching: Safe Attachment and Detachment 16

2.6 Dynamic Binary Instrumentation (dynamic binary instrumentation (DBI)) Architectures 18

2.6.1 Core Architectural Components 19

2.6.2 The Execution Lifecycle: The JIT Loop 19

2.6.3 The Client Interface: Instrumentation vs. Analysis Functions 21

2.6.4 High-Level Abstraction: Intel PIN 22

2.6.5 Bare-Metal DBI: DynamoRIO 23

Chapter 3 Related Work **25**

3.1 Sources 26

3.2 Style 26

Chapter 4 Methodology **28**

Chapter 5 Results **30**

5.1 Discussion 31

Chapter 6 Conclusion **32**

Chapter 1

Introduction

At its core, binary instrumentation is the technique of inserting additional code into an existing, compiled executable to observe or modify its runtime behavior without requiring access to the original source code. This capability is critical for enforcing security policies and monitoring performance.

Advisor: Devo ancora estendere questo capitolo

Chapter 2

Background

This chapter begins by introducing the foundational concepts of binary instrumentation within a Windows environment necessary to understand the proposed work. Subsequently, it analyzes state-of-the-art frameworks utilized for binary instrumentation. This overview is essential for contextualizing the compilation targets selected for our Domain-Specific Language (DSL), which are discussed in detail in Methodology (Chapter 4).

2.1 Windows Process Architecture and DLL Injection

As discussed in Introduction (Chapter 1), binary instrumentation allows us to change the runtime behavior of a binary. To achieve this in modern Windows environments, we must overcome several operating system constraints. Since every process operates within its own isolated virtual address space, a standard process cannot directly read or write the memory of another process. Therefore, to perform binary instrumentation, we must first find a way to inject and execute our custom instrumentation payload within the context (and memory space) of the target process. This is achieved through the use of dynamic-link librarys (DLLs).

While DLLs are traditionally used to share code libraries across multiple applications, they possess specific characteristics that make them the standard vehicle for Windows instrumentation:

1. **Shared Address Space:** When a DLL is loaded into a process, it is mapped directly into that process's virtual address space. This grants the DLL full access to read, write, and patch the memory of the host process.

2. **Automatic Initialization:** When a DLL is successfully loaded, the Windows operating

system (OS) loader automatically invokes its entry point function, `DllMain`. This allows the instrumentation framework to immediately execute its initialization sequence without requiring the host process to explicitly call any functions.

3. **Automatic Cleaning:** If the DLL is unloaded but the instrumentation code remains active in memory, subsequent executions will jump to an unmapped memory region, triggering an access violation. To ensure proper cleanup, when a DLL is successfully unloaded, the Windows OS loader automatically invokes its entry point function to execute the detachment logic.

   The following is the template code for an instrumentation DLL's entry point (`DllMain`):

   Code 2.1: DllMain code for attaching/detaching instrumentation

```
// Dll entry point
BOOL APIENTRY DllMain (HMODULE hModule,
  DWORD   ul_reason_for_call,
  LPVOID  lpReserved)
{
  switch (ul_reason_for_call)
  {
    case DLL_PROCESS_ATTACH:
      // Instrumentation framework-specific logic for injecting
         instrumentation
      break;

    case DLL_PROCESS_DETACH:
      // Instrumentation framework-specific logic for disabling
         instrumentation
      break;
  }

  return TRUE;
}
```

To force a remote, unaware process to load our instrumentation DLL, a technique known as **DLL Injection** is utilized. The most common and reliable method relies on the Windows application programming interface (API) functions `CreateRemoteThread` and `LoadLibrary`. The injector process allocates memory in the target, writes the path of the DLL into that memory, and spawns a thread (using `CreateRemoteThread`) pointing to `LoadLibrary`, which reads and processes the specified DLL path.

2.2 The Application Binary Interface (ABI) and Calling Conventions

Once the instrumentation DLL is injected, it must interact with the target process's existing functions. To do this safely without corrupting the target process, the instrumentation framework must strictly adhere to the Application Binary Interface (ABI). The ABI defines the low-level rules for how functions interact at the assembly level, including how parameters are passed to a function, how return values are retrieved, and which entity is responsible for cleaning up the stack (the caller or the callee). These rules are formalized as *Calling Conventions.*

In Windows x86 architectures, multiple calling conventions exist, such as:

- `__cdecl`: Arguments are pushed onto the stack from right to left, and the *caller* cleans the stack.
- `__stdcall`: The standard for the Windows API. Arguments are pushed right to left, but the *callee* cleans the stack.
- `__fastcall`: The first two arguments are passed in the `ECX` and `EDX` registers for speed, while the rest are placed on the stack.

Understanding these conventions is critical for binary instrumentation. If an injected payload intercepts a `__stdcall` function but is accidentally compiled as `__cdecl`, the stack pointer will not be properly restored, leading to immediate memory corruption and process termination.

2.3 Execution Context and CPU State Preservation

Building upon the constraints of the application binary interface (ABI), it is necessary to formally define what constitutes the execution context of a running process and why preserving it is paramount to process stability during any form of binary instrumentation.

At any given nanosecond, the exact state of an executing thread is represented by the data held within the CPU's internal registers. On an x64 architecture, this includes General Purpose Registers, the Instruction Pointer, and various status flags (e.g., `EFLAGS`). Collectively, these transient values are referred to as the *CPU state.*

When an instrumentation framework successfully intercepts the execution flow, it redirects the instruction pointer to the user's custom analysis payload. Because this payload is a compiled piece of software itself, it inevitably utilizes the CPU registers to perform its own calculations, memory allocations, or logging. This introduces a critical point of failure known as **register corruption**. If the custom payload overwrites the contents of a register that the original, interrupted application was actively using (for example, to store an intermediate loop counter or a critical memory pointer), that original data is permanently lost.

When the payload finishes its execution and attempts to restore the original execution flow, the application will blindly continue operating using the corrupted register values. Therefore, to guarantee process stability, the original CPU state must be flawlessly preserved by the instrumentation framework.

2.4 Paradigms of Binary Instrumentation

Modern instrumentation engines generally fall into two broad architectural paradigms:

- **Inline API Hooking:** this paradigm involves modifying the original execution flow via localized jumps. It patches specific functions in memory, redirecting execution to a custom payload before returning to the original flow.
- **Dynamic Binary Instrumentation (DBI):** rather than modifying the original binary in place, DBI engines physically copy, recompile, and execute instructions inside a localized sandbox known as a Code Cache.

2.5 Inline API Hooking Architectures

Hooking is the specific mechanism used to instrument a function. A hook allows the redirection of the execution flow from the original code to newly injected analysis code. A standard inline hook consists of three primary logical components:

1. **The Target:** the original function whose execution we wish to intercept. The goal is to patch the first 5 to 7 bytes of the target function with an unconditional jump (`JMP`) to a user-defined payload. These initial bytes are restored to the target function once the instrumentation is complete and the framework detaches from the host process.
2. **The Detour (or Payload):** the custom function injected by the instrumentation framework. This function receives the execution flow, inspects the target function's arguments or return value, performs the desired analysis or policy enforcement, and then dictates whether the execution should continue or be blocked.

3. **The Trampoline:** in the paradigm of inline API hooking, placing an interception branch (`JMP`) inherently corrupts the target function's prologue. To maintain execution continuity, all inline instrumentation frameworks must implement an Out-of-Line Execution Bridge, universally referred to as a **"Trampoline"**. Conceptually, the trampoline is a dynamically allocated, executable memory buffer that fulfills two architectural mandates: it securely relocates the displaced prologue instructions and unconditionally routes the instruction pointer back to the unpatched remainder of the target function.

The manner in which these frameworks handle the CPU state and the ABI constraints defines their underlying architecture.

2.5.1 Bare-Metal Hooking: Microsoft Detours and MinHook

Frameworks like Microsoft Detours and MinHook are specifically designed for Windows x86/x64 and utilize a **"bare-metal" routing** approach. In this model, the hooking library has zero awareness of the CPU state or context at runtime. Consequently, they strictly enforce the ABI. The developer must possess precise knowledge of the C/C++ signatures (argument types and return types) and calling conventions of the function being hooked. As shown in Listing 2.2, using these two frameworks requires a strict type definition of the target function pointer to avoid stack corruption:

Code 2.2: Detours/MinHook requires strict ABI compliance.

```
// Strict function pointer definition required by Detours
static int (WINAPI * TrueMessageBoxW)(HWND, LPCWSTR, LPCWSTR, UINT) =
    MessageBoxW;

// Strict function pointer definition required by MinHook
typedef int (WINAPI * MESSAGEBOXW)(HWND, LPCWSTR, LPCWSTR, UINT);
MESSAGEBOXW fpMessageBoxW = NULL;
```

Furthermore, to access the arguments correctly, the hooked version of the function must strictly adhere to the original signature:

Code 2.3: Strict ABI compliance even in the Hooked Function

```
// Detours (same for MinHook)
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR
   lpCaption, UINT uType)
```

Following these guidelines ensures the compiler generates the correct code to prevent stack and register corruption and to correctly access the arguments of the target function being monitored or modified.

The code structure for the hooked function is identical in both frameworks. In the following example, the goal is to intercept a target process's call to `MessageBoxW` and modify the displayed dialog with custom text (`customCaption`) and altered options (`modifiedType`) during the PRE-LOGIC phase. After this, the trampoline is called, the original function is executed with the modified arguments, and control is returned to the hooked version (POST-LOGIC). Here, we intercept the user's choice (`result`), forcing a "yes" outcome even if the user pressed "no", and return the final result to the target process:

Code 2.4: Detours/MinHook Hooked function structure

```
// Detours (same for MinHook)
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR
    lpCaption, UINT uType)
{
  // PRE-LOGIC
  UINT modifiedType = MB_YESNO;
  LPCWSTR customCaption = L"Hooked! Please choose:";

  // Call to the trampoline
  int result = TrueMessageBoxW(hWnd, lpText, customCaption,
      modifiedType);

  // POST-LOGIC
  if (result == IDNO)
  {
    std::cout << "[HOOK POST-LOGIC] Intercepted IDNO (No). Changing
        result to IDYES (Yes)!" << std::endl;
    result = IDYES;
  }
  else if (result == IDYES)
  {
    std::cout << "[HOOK POST-LOGIC] Intercepted IDYES (Yes). Nothing
        to do." << std::endl;
  }

  return result;
}
```

2.5.2 Context-Aware Routing: Frida-Gum

Frida-Gum is the underlying C library of the Frida instrumentation toolkit, representing a modern approach to cross-platform instrumentation (X86/X64, ARM/ARM64, MIPS). In contrast to bare-metal routing, Frida-Gum utilizes **"Context-Aware Routing."** When a hook is triggered, the framework jumps not directly to the custom payload, but into a dynamically allocated memory block containing instructions that save all CPU registers into a hardware-agnostic structure (`GumInvocationContext`) before invoking the payload. After the hooking callbacks execute, these saved values are used to restore the CPU state exactly as it was prior to the hook, preventing process crashes.

This context awareness normalizes the ABI and abstracts the target process architecture. The developer does not need to define strict C++ function signatures; arguments and return values are accessed and modified dynamically by their index, regardless of the target process architecture. Due to these architectural differences, the implementation significantly diverges from that of Detours and MinHook. The developer only needs to specify the `on_enter` and `on_leave` hooking callbacks: `on_enter` executes before the target function, while `on_leave` executes after it finishes.

Below is the Frida-Gum equivalent code for altering the behavior of `MessageBoxW`:

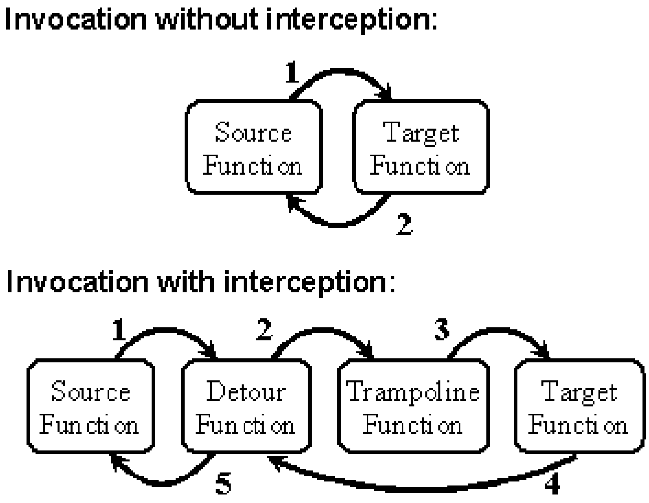
Code 2.5: Frida-Gum `on_enter/on_leave` for hooking MessageBoxW.

```
// Pre-logic
static void
example_listener_on_enter (GumInvocationContext * ic,
  gpointer user_data)
{
  // The arguments for MessageBoxW are 0-indexed:
  // 0: HWND hWnd
  // 1: LPCWSTR lpText
  // 2: LPCWSTR lpCaption
  // 3: UINT uType

  // Frida-Gum allows contextual argument extraction without strict C
      signatures
  guint originalType = GPOINTER_TO_UINT (
     gum_invocation_context_get_nth_argument (ic, 3));
  guint modifiedType = MB_YESNO;

  static const wchar_t* customCaption = L"Hooked, choose:";

  gum_invocation_context_replace_nth_argument (ic, 2, (gpointer)
     customCaption);
  gum_invocation_context_replace_nth_argument (ic, 3,
     GUINT_TO_POINTER (modifiedType));
}

// Post-logic
static void
example_listener_on_leave (GumInvocationContext * ic,
  gpointer user_data)
{
  int result = GPOINTER_TO_INT (
     gum_invocation_context_get_return_value (ic));
```

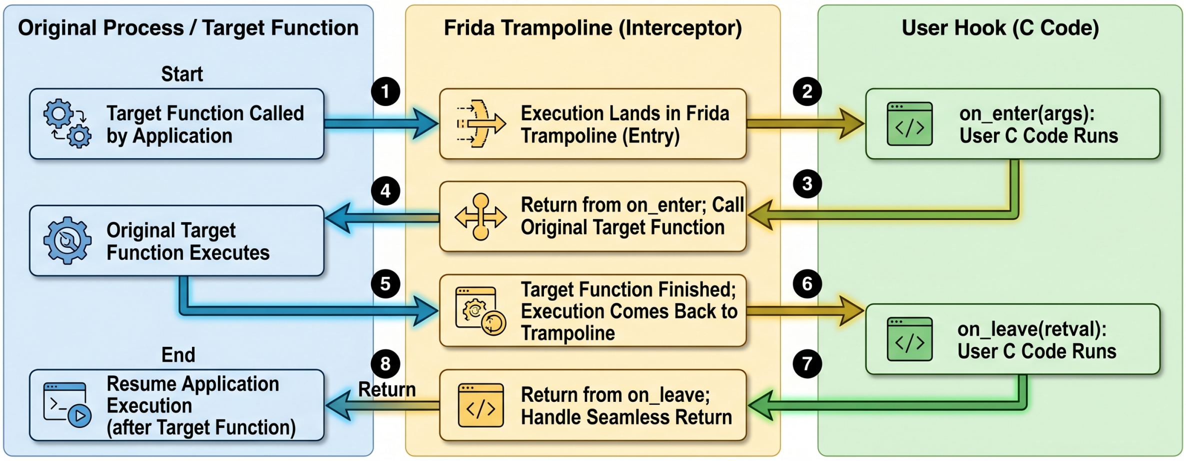
```
    if (result == IDNO)
    {
      g_print ("Intercepted IDNO (No). Changing result to IDYES (Yes)\n
         ");
      gum_invocation_context_replace_return_value (ic, GINT_TO_POINTER
         (IDYES));
    }
    else if (result == IDYES)
    {
      g_print ("Intercepted IDYES (Yes). Nothing to do\n");
    }
  }
```

2.5.3 Passive vs. Active Trampoline Architectures

While all the presented frameworks utilize inline prologue patching, their trampoline architectures are fundamentally opposed. Detours and MinHook utilize a **Passive Trampoline**, strictly acting as a repository for displaced instructions and forcing the user's hook to act as the primary execution controller (the developer's C++ code manually calls the Trampoline). Conversely, Frida-Gum utilizes an **Active Trampoline**: when the target function is intercepted, execution jumps directly into the trampoline, which assumes total control of execution routing and register preservation, as depicted in Figure 2.1:


Invocation without interception:
1
Source Function
Target Function
2
Invocation with interception:
1
2
3
Source Function
Detour Function
Trampoline Function
Target Function
5
4

(a) Detour/MinHook Execution Flow


Original Process / Target Function
Frida Trampoline (Interceptor)
User Hook (C Code)
Start
Target Function Called by Application
1
Execution Lands in Frida Trampoline (Entry)
2
on_enter(args): User C Code Runs
3
4
Return from on_enter; Call Original Target Function
Original Target Function Executes
5
Target Function Finished; Execution Comes Back to Trampoline
6
on_leave(retval): User C Code Runs
End
8
Return
7
Resume Application Execution (after Target Function)
Return from on_leave; Handle Seamless Return

(b) Frida-Gum Execution Flow

Figure 2.1: A comparison between the different architectures and roles of the Trampoline

2.5.4 Instruction Boundaries and the Role of Disassemblers

Advisor: Non so se mettere questa sezione

A fundamental challenge in implementing the hook structure described above, particularly on Intel x86 and x64 architectures, is the variable-length nature of the instruction set. To detour a target function, an inline hooking framework typically needs to overwrite the first few bytes of the function's prologue with an unconditional jump (`JMP`) to the custom payload. In a standard 32-bit architecture, this jump requires 5 bytes (1 byte for the opcode, 4 bytes for the relative address).

If the framework blindly overwrites exactly 5 bytes, it risks cutting an existing instruction in half. For example, if the original prologue consists of two 3-byte instructions, overwriting 5 bytes will corrupt the second instruction, leaving a hanging, invalid byte. When the trampoline attempts to execute these copied bytes to resume the program, the CPU will encounter an invalid opcode and crash the process.

To prevent this, hooking frameworks must incorporate a Length Disassembler Engine (LDE). Before applying any patches, the engine disassembles the target function's prologue to calculate exact instruction boundaries. It then copies whole, unfragmented instructions into the trampoline until it has safely relocated at least the number of bytes required for the jump (padding any remaining bytes in the original function with `NOP` instructions).

The choice of disassembler heavily impacts the footprint and dependencies of the framework:

- **Microsoft Detours** relies on a proprietary, internal instruction decoder optimized specifically for standard compiler prologues.
- **MinHook** utilizes a highly optimized, custom-built Hacker Disassembler Engine (HDE32/HDE64), as it only needs to calculate instruction lengths rather than fully analyze instruction semantics.
- **Frida-Gum** requires deep semantic analysis of the instructions to achieve its context-aware routing. To accomplish this, it statically links against Capstone, a comprehensive and heavy multi-architecture disassembly framework, significantly increasing the size of the resulting DLL.

2.5.5 Transactional Patching: Safe Attachment and Detachment

Another critical challenge in inline hooking is applying the memory patch (e.g., the `JMP` instruction) to a live, running process without causing a **race condition**. In a multi-threaded application, if one thread attempts to execute a target function at the exact moment the instrumentation framework is overwriting its prologue, the CPU may attempt to execute a partially written, invalid instruction.

To guarantee thread safety during the patching phase (both attaching and detaching), modern instrumentation frameworks implement a **transactional model**. Instead of patching memory directly and immediately, the developer queues the hooks. When the transaction is finalized, the framework briefly suspends all threads in the target process (except the one applying the patch), writes the bytes to memory atomically, flushes the CPU's Instruction Cache, and finally resumes the suspended threads.

Detours makes this transaction explicitly visible to the developer. The API requires opening a transaction, manually declaring which threads need to be stopped, attaching/detaching the specified hooks, and committing the transaction:

Code 2.6: Microsoft Detours transactional model for safe attachment and detachment.

```
// --- SAFE ATTACHMENT ---
// Start the transaction
DetourTransactionBegin();

// Enlist threads to be suspended during the patch
DetourUpdateThread(GetCurrentThread());
// here we specify possible other threads we want to stop...

// Queue the hooks
DetourAttach(&(PVOID&)TrueMessageBoxW, HookedMessageBoxW);
// here we specify possible other hooks we want to inject...

// Apply patches atomically, suspending the specified threads and
   flushing the instruction cache
DetourTransactionCommit();


// --- SAFE DETACHMENT ---
DetourTransactionBegin();

DetourUpdateThread(GetCurrentThread());
// here we specify possible other threads we want to stop...

DetourDetach(&(PVOID&)TrueMessageBoxW, HookedMessageBoxW); // Queue
   removal
// here we specify possible other hooks we want to detach...

DetourTransactionCommit();
// Suspends threads, restores original bytes, resumes
```

MinHook abstracts the explicit transaction start and end calls, but enforces the exact same synchronization internally when hooks are enabled or disabled (Listing 2.7):

Code 2.7: MinHook internal thread synchronization during enable/disable phases.

```
// --- SAFE ATTACHMENT ---
MH_CreateHook(&MessageBoxW, &HookedMessageBoxW, reinterpret_cast<
   LPVOID*>(&fpMessageBoxW));
// here we specify possible other hooks we want to inject...
// MH_EnableHook internally suspends threads, writes the JMP, and
    resumes
MH_EnableHook(&MessageBoxW);
//or MH_EnableHook(MH_ALL_HOOKS); if we have more than one hook

// --- SAFE DETACHMENT ---
// MH_DisableHook internally suspends threads, restores original
    bytes, and resumes
MH_DisableHook(&MessageBoxW);
//or MH_DisableHook(MH_ALL_HOOKS); if we have more than one hook
```

Frida-Gum employs an identical safety mechanism through its Interceptor API:

Code 2.8: Frida-Gum utilizing transaction boundaries for hook management.

```
// --- SAFE ATTACHMENT ---
gum_interceptor_begin_transaction (interceptor);

gum_interceptor_attach (interceptor,
  GSIZE_TO_POINTER (gum_module_find_export_by_name (user32, "
     MessageBoxW")),
  listener,
  NULL,
  GUM_ATTACH_FLAGS_NONE);
// here we specify possible other hooks we want to inject...

gum_interceptor_end_transaction (interceptor);
// Threads are paused and patches applied

// --- SAFE DETACHMENT ---
gum_interceptor_begin_transaction (interceptor);

gum_interceptor_detach (interceptor, listener);
// here we specify possible other hooks we want to detach...

gum_interceptor_end_transaction (interceptor);
// Threads are paused and bytes restored
```

2.6 Dynamic Binary Instrumentation (DBI) Architectures

For deeper execution analysis at the assembly level, DBI engines are utilized. Unlike inline hooking, which patches the original binary in place, DBI engines ensure that the target application's original code is never executed directly by the CPU. Instead, the framework employs a **Just-In-Time (JIT) compilation** mechanism to dynamically translate and instrument instruction sequences.

2.6.1 Core Architectural Components

Modern DBI frameworks rely on a unified architectural pattern comprising the following components:

1. **The Dispatcher (Control Flow Manager):** The dispatcher is the central component of the DBI engine, responsible for orchestrating the execution state. Because the application and the DBI framework share the same process but operate under different scopes, they must safely transit between "framework mode" and "application mode." The dispatcher trigger a context-switch mechanism that carefully saves and restores the application's state whenever execution exits or enters the code cache.
2. **Just in time (JIT) Compiler:** Instead of translating the entire binary before execution, these architectures translate the application's instructions dynamically in small fragments. During this translation phase, the framework decodes and encodes (into the framework's abstraction) the original instructions and has the opportunity to insert custom instrumentation code or patches before emitting the new sequence.
3. **The Code Cache (The Execution Sandbox):** To mitigate the massive performance penalty of repeatedly translating the same code, DBI frameworks employ a software-based code cache. The translated and potentially instrumented fragments are stored in this sandboxed memory region. When the application re-executes a previously translated section of code, it runs directly from the code cache, which significantly optimizes execution speed.
4. **The Client Interface (Instrumentation API):** This is the abstraction layer exposed to the developer. It provides the necessary functions to inspect the program state, traverse instruction sequences, and specify exactly where and how instrumentation should be injected.

2.6.2 The Execution Lifecycle: The JIT Loop

To formally describe how these components interact at runtime, we can outline the execution lifecycle of a standard block of code:

1. **Interception:** The target application attempts to execute a block of original code. The DBI engine intercepts this attempt and yields control to the Dispatcher.

2. **Cache Lookup:** The Dispatcher queries a fast-lookup hash table to check if the target memory address has already been translated. If it results in a cache hit, the CPU is redirected to the Code Cache.

3. **Translation (On Cache Miss):** If the code has not been encountered before, the JIT Compiler decodes original binary instructions one by one to form a straight-line sequence that terminates at a control transfer, such as a branch, call, or return.

4. **Instrumentation:** The engine queries the Client Interface to determine if the user wishes to instrument the block or a single instruction within it. If so, the instrumentation instructions are injected.

5. **Emission and Linking:** Most DBI frameworks use this phase to apply patches to the control-flow instructions at the end of the sequence, guaranteeing that when the block finishes running, it will redirect back to the framework rather than jumping to a native memory address. If the block ends in a direct branch to another previously translated block, the engine "links" them together directly in the cache (if it was an indirect branch, it cannot know the target before executing it), allowing the CPU to bypass the Dispatcher in future iterations. The modified block is emitted into the Code Cache.

6. **Execution:** The Dispatcher performs a context switch, restoring the application state, and commands the CPU to execute the newly cached block natively.

7. **Code Cache Exit:** Execution remains safely inside the code cache — often jumping directly from one translated block to another through "linking"—until it hits a boundary. This boundary might be an unlinked branch or an indirect branch whose target hasn't been translated yet. When this happens, the block executes an exit stub which saves the CPU's state and transfers control back to the Dispatcher, restarting the loop.

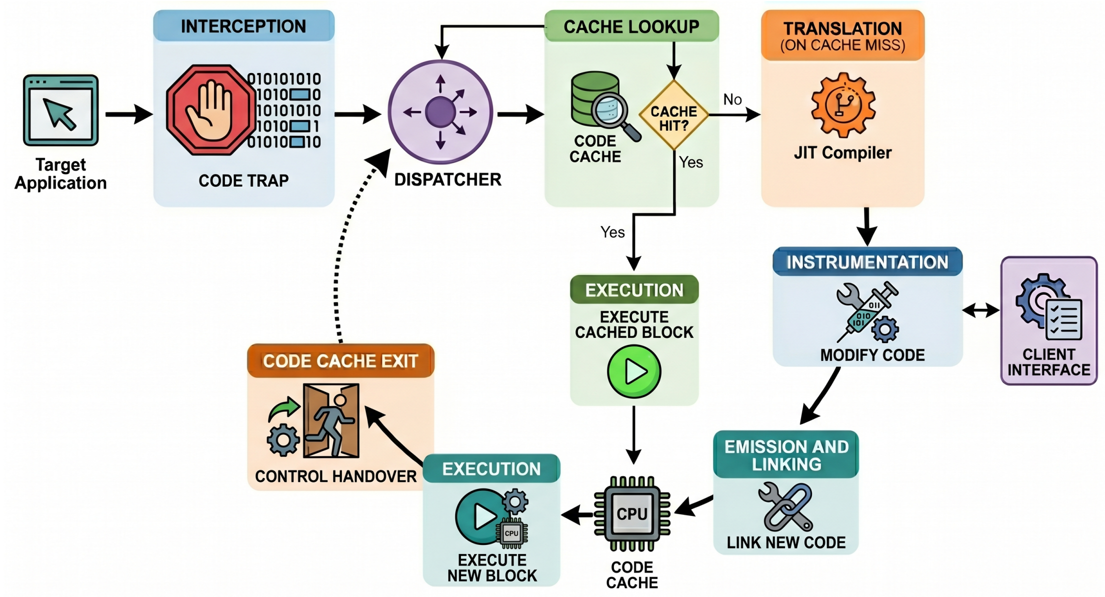

Target
Application
INTERCEPTION
CODE TRAP
DISPATCHER
CACHE LOOKUP
CODE
CACHE
CACHE
HIT?
No
Yes
TRANSLATION
(ON CACHE MISS)
JIT Compiler
Yes
EXECUTION
EXECUTE
CACHED BLOCK
INSTRUMENTATION
MODIFY CODE
CLIENT
INTERFACE
CODE CACHE EXIT
CONTROL HANDOVER
EXECUTION
EXECUTE
NEW BLOCK
CPU
CODE
CACHE
EMISSION AND
LINKING
LINK NEW CODE

Figure 2.2: The Execution Lifecycle of a DBI framework

While DBI frameworks allow a more granular analysis of the target code, they introduce **significantly higher overhead** compared to inline hooking. Every single instruction of the program needs to be JIT-compiled before execution, even if no instrumentation is injected.

2.6.3 The Client Interface: Instrumentation vs. Analysis Functions

DBI frameworks require the developer to interact with the engine using a **two-tiered callback paradigm**. The developer must supply two distinct types of functions to the engine, each operating at fundamentally different times during the execution lifecycle:

- **The Instrumentation Function (The "Where" and "What"):** This callback is invoked by the DBI engine *only once* per newly discovered sequence of instructions during the JIT compilation phase. Its purpose is to inspect the static properties of the instructions (e.g., opcodes, operands) and decide *if* and *where* custom code should be injected. If injection is required, it uses the API to queue the insertion of the analysis function.
- **The Analysis Function (The "Action"):** This is the actual custom payload injected into the Code Cache. Unlike the instrumentation function (executed only

at compile time), it is executed *every time* the CPU runs the corresponding instrumented block of code. For this reason, the analysis function must be heavily optimized and efficient.

The level of abstraction provided by the Client Interface significantly alters how developers interact with the engine.

2.6.4 High-Level Abstraction: Intel PIN

Intel PIN is a standard academic DBI framework with a proprietary (free for non-commercial use) license. It provides a heavily abstracted C++ API.

To utilize Intel PIN, developers do not modify the core engine itself. Instead, they write a custom C++ program known as a *PINtool*. A PINtool represents the physical implementation of the Client Interface discussed previously; it is the user-defined module that contains both the Instrumentation Functions and the Analysis Functions.

Architecturally, a PINtool is compiled as a standard shared library (a DLL on Windows). When the user launches a target application under PIN, the engine automatically injects the PINtool DLL into the target process's memory space. Once loaded, the engine invokes the PINtool's `main` function, allowing the tool to register its specific callbacks.

In the following PINtool, we can observe the granularity achievable using DBI: in this case, we can log conditional branches to determine which paths are taken during code execution.

Code 2.9: An example of a PINtool for branch logging

```
// Analysis function (Runs at execution time in the Code Cache)
void record_branch(ADDRINT instruction_address , ADDRINT
    target_address , BOOL taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function (Runs at JIT compilation time)
void trace(INS instruction , void *v) {

  // PIN abstracts away module resolution: we want to instrument
  // only target application instructions , filter possible external
      libraries
  RTN rtn = INS_Rtn(instruction);

  if (RTN_Valid(rtn) && IMG_IsMainExecutable(SEC_Img(RTN_Sec(rtn))))
      {
```

```
      // If it is inside the target executable, check if it's an IF
          branch
      if (INS_IsBranch(instruction) && INS_HasFallThrough(instruction))
          {
        // Inject analysis function right before the jump happens.
        INS_InsertCall(instruction, IPOINT_BEFORE, (AFUNPTR)
            record_branch,
          IARG_INST_PTR,
          IARG_BRANCH_TARGET_ADDR,
          IARG_BRANCH_TAKEN,
          IARG_END);
      }
    }
  }
```

2.6.5 Bare-Metal DBI: DynamoRIO

DynamoRIO is the open-source alternative that exposes a lower-level C API. The following code is the equivalent branch-logging example seen previously for PIN:

Code 2.10: DynamoRIO branch instrumentation example

```
// Analysis function
static void record_branch(app_pc instr_addr, app_pc target_addr, int
   taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function
static dr_emit_flags_t event_bb_analysis(void *drcontext, void *tag,
   instrlist_t *bb, instr_t *inst, bool for_trace, bool translating,
void *user_data) {

  // Manual module tracking to filter out OS libraries
  app_pc instr_addr = instr_get_app_pc(inst);

  module_data_t *mod = dr_lookup_module(instr_addr);
  bool is_main_exec = false;

  if (mod != NULL) {
    module_data_t *main_mod = dr_get_main_module();

    if (main_mod != NULL) {
      if (mod->start == main_mod->start) { is_main_exec = true; }
      dr_free_module_data(main_mod);
```

```
        }
        dr_free_module_data(mod);
      }

      if (!is_main_exec) return DR_EMIT_DEFAULT;

      // Check if it's a conditional branch (CBR)
      if (instr_is_cbr(inst)) {
        // Inject analysis function into the basic block (bb)
        dr_insert_cbr_instrumentation(drcontext, bb, inst, (void *)
            record_branch);
      }
      return DR_EMIT_DEFAULT;
    }
```

Using DynamoRIO, developers can directly manipulate the instruction flow. In PIN, developers generally "wrap" existing instructions (adding a call before or after an instruction) but cannot easily modify the instruction itself. In DynamoRIO, the developer is handed a linked list of instructions (`instrlist_t`) and can easily manipulate or modify them (e.g., reordering them or moving a load instruction earlier to test for race condition).

Furthermore, because PIN is designed to be highly opaque, using `INS_InsertCall` prompts PIN's JIT compiler to automatically analyze the custom analysis function for deciding which registers need to be saved and restored based on what the C++ code actually uses. With DynamoRIO, developers can manage the state and flags manually for specialized analysis.

In conclusion, while DynamoRIO may present a steeper learning curve than Intel PIN, it affords developers greater transparency and raw power, providing fine-grained control over the translated code and the code cache.

Chapter 3

Related Work

This chapter is dedicated to the work that has been done by others in the field of your thesis. Don't write a mere list of papers with a summary of each. Instead, the goal is to give a picture of how your work is situated in the context of the work of others. In this case, you should think your skeptical reader is asking "hasn't this been solved already?".

There will be two main kinds of work that you will need to discuss: similar work to yours, and work that is complementary to yours.

Similar work will try to solve the same problem you are addressing, or a similar one. Here, focus on the differences: for example, you may write something along the lines of "**DBLP:journals/x/Turing50** provides a naive definition of 'intelligent machine', which has this and that limitation. Our work is similar to that of **DBLP:journals/x/Turing50**, but we address these problems by doing this and that."[1]

Complementary work does not try to solve the same problem you are addressing, but it is important to get a full picture of your contribution. For example, if you are *not* attempting to solve a particular problem (it is "out of scope" for your work), but the solution to that problem is necessary for your work to be useful, you should explain that here.

The Related Work chapter may end up in other places of the document. Consider that it does two pieces of work at the same time: *(i)* substantiating the claim you made in the Introduction (Chapter 1) that your problem is relevant, and *(ii)* describe the context around your work to help understand its impact. If the latter part is more important, it may make sense to move this section to the end of the document, just before the Conclusion (Chapter 6). If the Background (Chapter 2) is not necessary to understand this chapter, it is also a good idea to move it just after the Introduction (Chapter 1).

---

[1] Do not be arrogant! I called the work of Turing naive as a joke. For sure your work will have limitations that other work may not have, and you should be honest about them.

3.1 Sources

While doing your review of related work, Google Scholar[2] is probably the easiest search engine for academic papers. You can be mindful of some bibliometric tricks to understand which papers are likely to be most influential in the field. You can see on Scholar the number of citations of a paper; CORE[3] and Scimago[4] provide rankings of conferences and journals respectively. Non-peer-reviewed articles like those published only on arXiv[5] may, of course, be important, but you should be more careful and skeptical about them. In case of doubt, ask for guidance from your advisor.

Another tool that you should *use with caution* is Semantic Scholar.[6] It provides you with large language model (LLM)-based tools to synthesize and ask questions about papers. Always double-check, because LLMs hallucinate and, as usual, you take responsibility for whatever you write in your document: treat LLMs as a useful tool you shouldn't trust.

Wikipedia is a great source of information, but you shouldn't consider it as reliable. Wikipedia articles should always provide the sources of the information they contain; you should check it yourself, judge its reliability, and cite the source.

In general, be careful about citing websites: make sure that they're authoritative, and when it's not obvious, justify why you consider them to be worth citing. Unlike this document, which I don't expect readers to print, your thesis must be readable in printed form: hence, make URLs readable in the text.

If a paper is very important for your own piece of work, have a look at the papers that cited it, to see if there are more recent works that are relevant to your thesis.

3.2 Style

Here are a few tricks about how to cite other pieces of work:

- If a paper is available in a peer-reviewed venue (journal or conference), cite that version rather than the non-reviewed one. Sometimes papers are published in two similar versions: one in a conference and an extended one in a journal. In that case, cite the journal.

---

[2] `https://scholar.google.com`

[3] `https://portal.core.edu.au/conf-ranks/`

[4] `https://www.scimagojr.com`

[5] `https://arxiv.org`

[6] `https://www.semanticscholar.org/`

- Use citations such as “The Hamiltonian cycle problem is NP-complete [**DBLP:conf/coco/Karp7** or “**DBLP:conf/coco/Karp72** showed that the Hamiltonian cycle problem is NP-complete”. Do not write “[**DBLP:conf/coco/Karp72**] showed that the Hamiltonian cycle problem is NP-complete”. Note, like in this case, that academics pretty much always refer to papers by citing authors’ names rather than their titles.[7]

- DBLP[8] is a good source of quality Bibtex records for papers you may want to cite.

- You may have a doubt about whether to put a link in the text or use a reference in the bibliography. Use a reference if it is a document you want to refer to (even if it’s not a text document, like a video or audio), use a link if you just want to refer to something but do not use it as a reference document.

- If you decide to leave links, make it so that they’re usable even in printed form. The `\myhref` command of this document can be an example of how to do it.

---

[7] If you use LaTeX, check the source code for how to cite paper authors using the `\textcite` command.

[8] `https://dblp.org`

Chapter 4

Methodology

This chapter describes the core of your work: the methodology you used to solve the problem you described in the Introduction (Chapter 1). It is quite likely to have a different name (e.g., the name of the tool you wrote), and/or to be split into multiple chapters. *Methodology* suggests that you will spend space discussing your overall approach and justification for it: it is a good idea for a Master's Thesis, as opposed to just a list of things that you have done (which could be more correctly called *Method* or *Methods*, and would risk being boring and not very useful for the reader).

This chapter will likely be quite densely connected with things you have written in the Background (Chapter 2), and the results that you will show in the Results (Chapter 5). Put abundant references to help the reader navigate the document.

Try to keep a level of detail such that a skilled reader will be able to reproduce your result, but do not include unnecessary details that the reader can find out for themselves (e.g., the list of commands to perform a standard task). That is indeed valuable information that can help reproducibility, but if you do not think it is needed to understand your work, its place is likely to be in an appendix or in the documentation of the code you wrote, if you decide to release it.

This chapter should include work that has been done by you. If that is not the case, consider moving it to the Background (Chapter 2), or make it very clear that you are not the author. The same thing applies to images: if they aren't yours, you should cite the source.

The results of your work should generally not be included in this chapter. In some cases, preliminary results can be included to explain why you took a certain direction in your work (e.g., you took a choice rather than another because you measured that it was more efficient), but the overall evaluation of your work should be in the Results (Chapter 5).

You are likely to include algorithms in pseudocode. Check relevant LaTeX packages[1] for that.

---

[1] https://www.overleaf.com/learn/latex/Algorithms

Chapter 5

Results

Here you will present the results of your work. Your skeptical reader is now asking themself "is this working?". You will show here to what extent it does. Be honest about the limitations: if your system doesn't work in some cases, you should say so (and explain where and why, if you can). Like the Methodology chapter, this one may end up being split into multiple chapters.

Keep in mind that this shouldn't be a mere dump of experimental results; you're rather *teaching* your reader how and to what extent you managed to solve the problem you described in the Introduction (Chapter 1). If you have additional results that may be useful but are not necessary to understand the points you're making (e.g., you evaluated your system on multiple datasets and the results all tell the same story), the place for them is in an appendix.

This chapter should have a lot of links to the methodology chapter(s), because you're evaluating the choices you made there. If you developed a system made of multiple parts, make sure that you test them separately and together, so that the reader can understand how important each part is.

This chapter is likely to be quite full of figures and tables. Try to make them as informative as possible (e.g., use multiple lines in the same plot if possible). Showing graphs effectively is a complex art; try to spend some time on it and ask for guidance from your advisor. Whenever you have a figure or a table, make sure that you refer to it in the text (after all, if it's not referred to in the text it means it has no part in the story you're telling, so it has no place in this Chapter). Always use the text (and the caption, if you can) to explain what you want the reader to understand from the figure.

If you are using LaTeX, it will automatically place figures, tables, algorithms, etc. somewhere in the text. These parts of the documents are called *floats* because their position

Column 1	**Column 2**	**Column 3**
aaa	2	3.42
bbb	505	6.00
ccc	8	901.02

Table 5.1: A table using the LaTeX `booktabs` package. Note there are no vertical rules. In case you want to do more fancy stuff (e.g., merging cells), check also the `multirow` and `multicol` packages. It's generally a good idea for readability to align text to the left and numbers to the right (use the same number of significant digits).

floats around depending on LaTeX's will. There are options to advise LaTeX about where floats should be placed. My advice is not to waste too much time on this and trust that they will end up in a good place. Generally, write the floats' code before you refer to them: they will never be placed before the place they appear in.

For Figures, generally use vectorial formats (e.g., PDF, SVG) where it makes sense if you can. They will result in higher-quality images that are in most cases also smaller, leading to shorter compiling times and a smaller resulting PDF.

Tables look better without vertical rules: an example is in Table 5.1.

5.1 Discussion

At the end of this chapter, take a step back and summarize all the results so that the reader can understand the big picture. Sometimes, this becomes large and important enough to be worth a chapter on its own.

Chapter 6

Conclusion

You are finally done! Here you will summarize for the reader what you have taught them through the document, and the main takeaways of your work you would like them to remember.

It is also a good idea to discuss the limitations of your work and your views about what can be possible future work.

Bibliography

[BGA03] Derek Bruening, Timothy Garnett, and Saman P. Amarasinghe. "An Infrastructure for Adaptive Dynamic Optimization". In: *1st International Symposium on Code Generation and Optimization (CGO 2003), 23-26 March 2003, San Francisco, CA, USA*. IEEE Computer Society, 2003, pp. 265–275. DOI: `10.1109/CGO.2003.1191551`. URL: `https://doi.org/10.1109/CGO.2003.1191551`.

[Fri23] Frida Developers. *Frida Documentation: C API (Frida-Gum)*. 2023. URL: `https://frida.re/docs/c-api/`.

[Fri26] Frida Developers. *Frida-Gum: Cross-platform instrumentation and introspection library*. 2026. URL: `https://github.com/frida/frida-gum`.

[HB99] Galen C. Hunt and Doug Brubacher. "Detours: Binary Interception of Win32 Functions". In: *Proceedings of the 3rd USENIX Windows NT Symposium, Seattle, WA, USA, July 12-13, 1999*. USENIX Association, 1999, pp. 135–143. URL: `https://www.cs.columbia.edu/~junfeng/10fa-e6998/papers/detours.pdf`.

[Int26] Intel Corporation. *Pin - A Dynamic Binary Instrumentation Tool*. 2026. URL: `https://software.intel.com/sites/landingpage/pintool/docs/99776/Pin/doc/html/index.html`.

[Kag26] Tsuda Kageyu. *MinHook: The Minimalistic x86/x64 API Hooking Library for Windows*. 2026. URL: `https://github.com/TsudaKageyu/minhook`.

[Luk+05] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. "Pin: building customized program analysis tools with dynamic instrumentation". In: *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2005, Chicago, IL, USA, June 12-15, 2005*. Ed. by Vivek Sarkar and Mary W. Hall. ACM, 2005, pp. 190–200. DOI: `10.1145/1064978.1065034`. URL: `https://doi.org/10.1145/1064978.1065034`.

[Mic26] Microsoft Corporation. *Detours: Software package*. 2026. URL: `https://github.com/microsoft/Detours`.

[The26] The MITRE Corporation. *Process Injection: Dynamic-link Library Injection, Sub-technique T1055.001*. 2026. URL: `https://attack.mitre.org/techniques/T1055/001/` (visited on 05/15/2026).